

The LLM Wiki Pattern

A bonus resource inspired by Andrej Karpathy — companion to the Build Your Second AI Brain lesson

A note on the source. This bonus guide is inspired by a public gist written by **Andrej Karpathy** titled **LLM Wiki**, where he describes a pattern for building personal knowledge bases that LLM agents actively maintain for you. Everything you're about to read is my own interpretation of his ideas, adapted specifically for the Obsidian vault setup you built in the lesson. All credit for the original thinking belongs to him. I strongly recommend reading the original gist directly — it's short, well written, and sparked a whole wave of community implementations worth exploring.

Original gist: [github.com/karpathy](https://gist.github.com/karpathy) — search for *llm-wiki.md*

Gist ID: 442a6bf555914893e9891c11519de94f

Why I made this bonus guide

When I saw Karpathy's gist, I was struck by how closely his ideas lined up with what we just built together in the Obsidian lesson. He describes a pattern where an AI agent doesn't just search your documents when you ask questions — it actively builds and maintains a living wiki of everything you know, updating cross-references, flagging contradictions, and growing the knowledge base with every new source. That is almost exactly what a well-used Obsidian vault paired with your OpenClaw agent turns into over time.

The gist itself is intentionally abstract. Karpathy describes the pattern but leaves the implementation details to the reader — he explicitly says the right way to use his document is to share it with your own agent and work together to build a version that fits your needs. So that's what this bonus guide does for you. I've translated his abstract ideas into concrete prompts you can copy and paste straight into your chat with your agent, all built around the exact vault structure we set up in the main lesson.

If you haven't read the original gist yet, I recommend doing so after you finish this guide. It's a short read and it'll give you the deeper 'why' behind the prompts I'm about to hand you.

Who is Andrej Karpathy?

In case the name doesn't ring a bell, Andrej Karpathy is one of the most influential voices in modern AI. He was a founding member of OpenAI, later became head of AI at Tesla where he led the Autopilot vision team, and has since returned to independent research and education. His YouTube channel and lecture series on how large language models work have been watched by millions of developers and are widely considered some of the best free resources in the field.

When someone like Karpathy publishes a short pattern for how to build personal knowledge bases with LLMs, and thousands of developers immediately start implementing their own versions, it's worth paying attention. The pattern he describes isn't academic theory — it's based on how he personally uses LLMs to organise his own reading, research, and thinking.

The big idea in plain English

Most people use LLMs with documents in a fairly simple way. You upload a file or paste some text, ask a question, and the model finds the relevant parts and answers. That works, but there's a hidden problem: nothing accumulates. Every time you ask a question, the model starts from scratch. There's no memory, no growing understanding, no synthesis across everything you've ever read.

Karpathy's pattern flips this around. Instead of using the LLM as a question-answering machine that operates on raw documents, you use it as a **wiki maintainer** that builds and updates a structured, interlinked knowledge base sitting between you and your sources. When you add a new source, the agent doesn't just save it — it reads it, extracts the key information, writes a summary, updates related pages across the vault, flags contradictions with what you already knew, and logs what just happened.

The wiki becomes a compounding artifact. Every source you add makes the whole thing richer. Every question you ask can be added back as a new note so your explorations accumulate too. Over weeks and months, you end up with a personalised, deeply cross-referenced knowledge base that reflects everything you've read and thought about — and the maintenance cost is near zero because the agent does all the tedious bookkeeping.

The key insight: the tedious part of maintaining a knowledge base was never the reading or the thinking — it was the bookkeeping. Updating cross-references, keeping summaries fresh, noting when new data contradicts old claims. Humans abandon personal wikis because the maintenance burden grows faster than the value. Your agent doesn't get bored, doesn't forget, and can touch fifteen files in a single pass.

The three layers of the pattern

Karpathy breaks the whole system into three clear layers. Understanding these three layers is the most important part of this guide, because once you see them, the prompts in the next section will make intuitive sense.

Layer 1 — Raw sources

This is your immutable collection of original materials. Articles you've read, PDFs of papers, transcripts from podcasts, notes from meetings, screenshots, whatever you want to feed into the system. The rule is simple: the agent reads from this layer but never modifies it. In our Obsidian setup, the **Inbox** folder is where raw sources land first, and once they've been processed they can move to more permanent homes.

Layer 2 — The wiki itself

This is where the agent lives and works. Summary pages, entity pages (one per person, one per project, one per concept), comparison notes, synthesis notes. The agent owns this layer entirely — it creates new pages, updates old ones, maintains links between them, and keeps everything consistent. You read it, the agent writes it. In our Obsidian setup, this is the **People, Projects, Resources, Business, and Daily** folders together — the whole cross-referenced web of notes that grows over time.

Layer 3 — The schema

This is a permanent set of instructions that tells the agent how the wiki is structured, what the conventions are, and how to behave when ingesting sources or answering questions. It's what turns a general-purpose chat model into a disciplined wiki maintainer. In our OpenClaw setup, this lives in two places: in the rules you already saved to your agent's permanent memory in the main lesson (pull before reading, push after writing, respect the six folders), and in any extra workflow rules you'll add using the prompts later in this guide.

How this maps to what you already built. The entire setup from the main lesson is a working implementation of Karpathy's three-layer pattern. Your Inbox folder is the raw layer. Your five other folders plus wikilinks form the wiki layer. Your permanent rules in your agent's memory are the schema layer. Everything else in this bonus guide is about extending that foundation with more sophisticated workflows.

The four operations — with ready-to-use prompts

Karpathy identifies three core operations in his gist: Ingest, Query, and Lint. I've added a fourth one called Schema Update because it's useful as a standalone operation in our setup. The four operations together form a complete workflow that you can use daily with your vault. Every prompt below is ready to copy and paste straight into your chat with your agent.

Operation 1 — Ingest

Ingest is what happens when you add something new to your vault — an article you read, notes from a meeting, a podcast transcript, a document a client sent you. The goal isn't just to dump the source into the vault. It's to have the agent read it carefully, pull out the important information, file it in the right places, update related notes across the vault, and log what just happened.

A single ingest can touch ten or fifteen different notes. A new article about a client might update their People page, add a summary to Resources, update the related Project note, add an entry to today's Daily note, and flag a contradiction with something you wrote last week. That's the kind of work that's impossible to do by hand but trivial for an agent.

How to use ingest

Drop the new source into your Inbox folder as a markdown note. The filename should be descriptive — something like *article-about-client-retention.md* rather than *untitled.md*. Then send your agent this prompt, adjusting the filename to match what you just added:

Please run an ingest pass on the new source I just added to my Obsidian vault at Inbox/[filename].md. Follow this workflow carefully:

- 1. Run git pull to make sure you have the latest version of the vault.*
- 2. Read the source carefully. Identify the key information, main claims, people mentioned, projects referenced, and any connections to things that already exist in my vault.*
- 3. Write a concise summary note in the Resources folder. Give it a descriptive filename and include wikilinks to any people, projects, or other concepts it connects to.*
- 4. For each person mentioned in the source, either create a new note in the People folder or update an existing one with the new information. Add wikilinks to the summary you just wrote.*
- 5. For each project mentioned, update the corresponding Projects note if one exists or flag it for me if one doesn't.*
- 6. Check whether anything in the new source contradicts or updates information in my existing notes. If it does, flag those contradictions clearly at the top of the summary note so I can review them.*
- 7. Append a short entry to today's note in the Daily folder explaining what you just ingested and which notes you touched. If today's daily note doesn't exist yet, create it.*
- 8. Commit and push everything with a meaningful commit message that mentions the source you ingested.*
- 9. Tell me a short summary of what you did and which notes were updated so I can review them.*

Operation 2 — Query

Query is what happens when you ask a question that requires pulling information from multiple notes across the vault. A basic question like 'what did I write about pricing' is simple, but real questions are usually more interesting — 'what do my notes suggest about why my retention dropped last quarter', or 'which of my projects are actually aligned with my stated business goals', or 'what's the through-line across everything I've read about AI agents this month'.

The key insight Karpathy makes is that these synthesised answers are valuable and shouldn't disappear into chat history. When your agent works hard to piece together a thoughtful answer from fifteen different notes, that answer should be saved back into the vault as a new note. This way your explorations compound over time — next month when you ask a similar question, the agent doesn't have to rebuild the synthesis from scratch. It already exists.

How to use query

Whenever you ask a question that might involve pulling from multiple notes, use this prompt pattern instead of just asking directly. It takes a few seconds longer but the output accumulates in your vault instead of vanishing:

I want to explore a question that requires synthesising information from multiple notes in my vault. The question is: [your question here].

Please follow this workflow:

- 1. Run git pull to get the latest vault state.*
- 2. Search the vault for all notes that might be relevant. Check across Projects, Resources, Business, People, and Daily folders. Don't limit yourself to the obvious matches — include anything that touches the question even tangentially.*
- 3. Read each relevant note carefully and extract what it says about the question.*
- 4. Write a synthesised answer that pulls together what you found. Cite specific notes using wikilinks so I can trace your reasoning back to the sources. If different notes say different things, surface the disagreement rather than smoothing it over.*
- 5. Save the synthesis as a new note in the Resources folder with a descriptive filename that captures the question. Include a date at the top so I know when this synthesis was made.*
- 6. Commit and push the new note with a meaningful commit message.*
- 7. Show me the synthesis in chat as well, so we can discuss it if I want to go deeper.*

Operation 3 — Lint

Lint is a health check on the whole vault. You don't run it every day — maybe once a week, or whenever you feel like the vault is getting messy. The agent scans everything and looks for problems:

notes that contradict each other, claims that newer sources have made obsolete, orphan notes that aren't linked from anywhere, important concepts that are mentioned in multiple places but don't have their own dedicated note, missing cross-references, and gaps where more research would help.

This is the operation that keeps the vault healthy as it grows. Without regular linting, a vault eventually drifts out of alignment with itself. Old notes stop matching new ones, cross-references break, orphans accumulate. A lint pass catches all of that and gives you a prioritised list of things to fix or investigate.

How to use lint

Please run a health check on my entire Obsidian vault and report what you find. Follow this workflow:

1. Run git pull to make sure you have the latest state.

2. Scan all notes across all six folders and look for the following issues:

— Contradictions: notes that claim different things about the same topic. Flag these clearly with both note names so I can decide which version is right.

— Stale claims: notes that contain information a newer source has updated or contradicted. These should probably be marked as outdated or revised.

— Orphan notes: notes that exist in the vault but aren't linked to from anywhere else. These are usually either drafts I forgot about or notes that should be connected but aren't.

— Missing concept pages: topics, people, or projects that get mentioned across multiple notes but don't have their own dedicated page in the right folder.

— Missing cross-references: pairs of notes that clearly relate to each other but don't link to each other. Suggest the wikilinks that should be added.

— Knowledge gaps: topics where my notes reference something but never actually explain it. These are good candidates for further research.

3. Don't fix anything automatically. Instead, give me a prioritised list of what you found, ordered by how important each issue is. I'll decide what to fix and tell you what to do next.

4. At the end, suggest a few good questions or new sources that would help fill the gaps you identified.

Operation 4 — Schema update

The schema is the set of rules your agent follows when it touches the vault. You already set up the basic schema in the main lesson — pull before reading, push after writing, respect the six folders. But as you use the vault for real work, you'll discover new workflow rules that would be useful to have permanently applied. Maybe you want your agent to always add a date header to new daily notes. Maybe you want it to tag meeting notes with the client name in the frontmatter. Maybe you want it to ask before deleting anything.

A schema update is simply adding a new permanent rule to your agent's memory. The key word is **permanent** — the rule should persist across every future conversation, not just this one session. That way the schema co-evolves with how you actually use the vault.

How to use schema update

Whenever you notice a pattern that would be useful as a permanent rule, use this prompt to save it to memory:

I want to add a new permanent rule to how you interact with my Obsidian vault. This rule should apply in every future conversation, not just this one.

The new rule is: [describe the rule clearly and specifically]

Please save this rule to your permanent memory alongside the existing vault rules (pulling before reading, pushing after writing, respecting the six folders). When you save it, confirm the full updated list of rules so I can double-check nothing got lost. From this point forward, you should follow this new rule automatically whenever it applies.

Examples of useful schema rules to consider

- Always add a date in ISO format at the top of any new note you create
- When creating a new meeting note, add a frontmatter field with the attendees
- Never delete anything without asking me first, even if I tell you to clean up
- When I ask about a person, always check their People note before answering
- When I start a new project, create a dedicated note in Projects and link it to my Business notes
- When you make a claim based on a note, always include a wikilink to that note
- When two notes contradict each other, always flag it before answering my question

Putting it all together — a weekly rhythm

You don't need to do all four operations every day. A rhythm that works well for most people looks something like this:

Daily

Whenever you read something interesting or have a useful conversation, drop the source into your Inbox and run an ingest. This is the most frequent operation and it's what makes the vault grow. Aim for one or two ingests a day when you're actively reading or working.

A few times a week

Whenever you have a question that requires pulling together information from multiple notes, use the query operation instead of just asking directly. This turns your thinking time into permanent vault content and makes next week's questions easier to answer.

Once a week

Run a lint pass on Friday afternoon or Sunday evening. Give yourself twenty minutes to go through what your agent found and fix the most important issues. This keeps the vault healthy and surfaces research gaps you can fill in the week ahead.

As needed

Whenever you notice a workflow pattern that would be useful to have permanent, run a schema update. These happen less often but they're the operations that really shape how your agent thinks about your vault over time.

Community implementations worth exploring

One of the most interesting things about Karpathy's original gist is the comment section. Within days of publishing, dozens of developers posted their own implementations of the pattern, each with a slightly different twist. If you want to go deeper, a few of these are worth checking out:

- **SwarmVault** — a full CLI tool that implements the pattern with support for many source types including transcripts, Slack exports, and EPUBs. Active development and interesting design decisions around guided ingest sessions.
- **Knowledge Forge** — a lightweight Node.js implementation with a clean web UI, good for learning how the pattern works under the hood.
- **OmegaWiki** — a research-focused version built by a team at Peking University that extends the pattern into a full research pipeline with experiment tracking.
- **Continuity** — an MCP server implementation so you can use the same knowledge base across multiple AI tools (Claude Code, Cursor, and others).

Most of these are built for developers and assume you're comfortable with the command line. They're not replacements for your Obsidian vault — they're alternative approaches to the same underlying idea. Look at them for inspiration, borrow patterns you like, but don't feel pressured to switch tools.

The Obsidian setup you built in the main lesson is a perfectly valid implementation of the pattern, and for most people it's the best starting point because the visual nature of Obsidian makes it easier to see what's actually happening as the wiki grows.

My take as the course creator

I wanted to end this bonus guide with a few honest thoughts from me, because I think there's something worth saying beyond just 'here are the prompts, have fun'.

The reason I put this bonus guide together in the first place is that when I read Karpathy's gist, I realised it was the clearest articulation I'd seen of something I'd been trying to teach throughout this whole course — the idea that a well-used AI agent isn't just a chatbot you talk to, it's a teammate that actively builds and maintains a knowledge base for you in the background. That's a mindset shift, and it's not obvious until someone draws the picture for you.

What I hope you take away from this bonus guide is not just the four prompts, though the prompts are useful. What I really hope you take away is the idea that your Obsidian vault and your agent together form a system that gets more valuable the more you use it. Every source you ingest, every question you turn into a synthesis note, every rule you add to the schema — all of it compounds. After three months of using the system the way this guide describes, you'll have something that feels genuinely like a second brain. After a year, you'll have something you'd be miserable without.

Start small. Don't try to implement all four operations on day one. Pick ingest first, use it for a week or two until it feels natural, then add query when you find yourself asking questions that span multiple notes. Lint comes later, when your vault is big enough to actually need a health check. Schema updates come naturally as you use the system and notice workflow patterns worth making permanent.

And please, read Karpathy's original gist. What I've given you in this guide is my practical adaptation for OpenClaw and Obsidian, but the original has context, nuance, and a community of thoughtful people in the comments that I couldn't possibly capture here. Think of this bonus guide as an introduction and the gist as your next step.

Full credit and attribution. Everything in this bonus guide is built on ideas Andrej Karpathy shared publicly in his LLM Wiki gist. The three-layer architecture, the three core operations (Ingest, Query, Lint), and the overall framing of the agent as a wiki maintainer all come from his original thinking. My contribution is adapting those ideas into concrete, copy-paste-ready prompts for the Obsidian plus OpenClaw setup you built in the main lesson. All credit for the original pattern belongs to him.

Read the original: search GitHub Gists for **karpathy llm-wiki**, or look up the gist ID **442a6bf555914893e9891c11519de94f**. It's short, well written, and has an incredible comment section with more than twenty community implementations you can explore.